

InvestAI Trading Platform

Steps to Go Live — staged runbook

Generated 2026-06-22 | branch merged to **main** (083145d)

[**STATUS**] Code remediation is **complete and merged to main**: 100 tests green, CI green, tree probabilities calibrated, live-venue trading gated to live mode, and the production Docker image dry-run verified end-to-end (build → migrate → serve → DB+Redis → /healthz 200). What remains is operational: the Railway deploy, then the edge and paper-soak gates below.

[**VERDICT**] **Paper-trade only after Stage 1.** Do **not** risk real money until Stage 2 (proven edge) **and** Stage 3 (paper soak) both pass. Stages are sequential — do not skip.

[**DONE**] verified in dev [**YOU RUN**] operator action [**HARD GATE**] must pass before proceeding

Stage 0 — Code correctness [**DONE**]

- 100 unit/integration/e2e tests green; ruff clean; CI (lint + pytest) in place.
- Migrations verified on real PostgreSQL **and** real TimescaleDB (migration 0002 builds 4 hypertables + retention; safe no-op on plain Postgres).
- Full pipeline verified over real Redis (rebalance → risk → execution → fill → risk feedback).
- Tree-model probabilities calibrated; auth path verified on real Postgres.
- Frontend login/auth gate, FE↔BE paths reconciled, production nginx build verified (tsc strict + vite).
- Safety invariant: paper/backtest mode **cannot reach a live venue** (gated at broker wiring, router, execution-engine hard guard, and CCXT sandbox).

Stage 1 — Deploy + smoke test on real infra [**YOU RUN**]

Part A — what only you can do (identity / secrets)

- 1 Authenticate the Railway CLI on this machine: `railway login` (opens a browser).
- 2 Rotate keys are already done; at deploy time put the **5 secrets** into Railway's secret store (the local `.env` is never shipped): `ALPACA_API_KEY`, `ALPACA_SECRET_KEY`, `BINANCE_API_KEY`, `BINANCE_SECRET_KEY`, `JWT_SECRET`.

Part B — provision & deploy

- 1 Create a Railway project; add **Postgres + Redis**. For TimescaleDB hypertables use a TimescaleDB image/template; on vanilla Postgres, migration 0002 safely no-ops.
- 2 Create **two services from this repo** (same Dockerfile): **api** (Dockerfile CMD `uvicorn api.main:app`; healthcheck `/healthz`; `releaseCommand = alembic upgrade head` runs automatically) and **worker** (override start to `python -m services.worker`).
- 3 Set environment variables on both services (table below).
- 4 Deploy. The release command runs migrations; the api service starts and the healthcheck must go green.

Variable	Value / source
DATABASE_URL	<code>{{Postgres.DATABASE_URL}}</code> (app converts to asyncpg itself)
REDIS_URL	<code>{{Redis.REDIS_URL}}</code>

ALPACA_API_KEY / ..._SECRET_KEY	rotated secret (Railway secret store)
BINANCE_API_KEY / ..._SECRET_KEY	rotated secret (Railway secret store)
JWT_SECRET	rotated 64-char secret (refused if default in live mode)
ALPACA_BASE_URL	https://paper-api.alpaca.markets (paper)
TRADING_MODE	paper
INITIAL_CAPITAL	100.00

Part C — verify the deployment

- 1 Create a login user: `railway run python scripts/create_user.py you@x.com '<pw>' admin`
- 2 Smoke test the pipeline: `railway run python scripts/smoke_test_pipeline.py` → must print `SMOKE TEST PASSED`.
- 3 Sanity: `GET /healthz 200`; `GET /api/v1/health` shows `db+redis ok`; `POST /api/v1/auth/token` returns a JWT; an authed request succeeds.

Already proven locally (deploy dry-run): the exact production image builds, runs 0001+0002 (4 hypertables), serves, connects DB+Redis, and answers `/healthz 200`, `/api/v1/health {database:ok, redis:ok}`, `/api/v1/auth/token 401` on bad creds. A root `.dockerignore` keeps `secrets/venv/git` out of the image (5.07GB → 2.87GB).

Stage 2 — Prove the strategy has edge [HARD GATE]

- 1 **Ingest real history for a small universe** (harness reads a `c`lose column; provider auto-selected by symbol, both keyless):
`python scripts/fetch_history.py AAPL --start 2015-01-01 --out aapl.csv`
... repeat for MSFT, SPY, and/or BTC/USDT.
- 2 **Run the gate across the whole universe at once** (net of costs):
`python scripts/validate_model.py aapl.csv msft.csv spy.csv --cost-bps 5`
(prefix `DYLD_LIBRARY_PATH=/opt/homebrew/opt/libomp/lib` on macOS; `libgomp1` is already in the Docker image).

[GATE] Per symbol: hit-rate > 52% AND Sharpe > 0.5 AND stability >= 0.75 (net-of-cost, non-overlapping holding periods). **Overall:** a green light needs a majority of **3+ symbols** to pass — a single-symbol pass prints *necessary but NOT sufficient*. The null self-test (zero-drift random walk, run with no args) must report **NO demonstrable edge**. Anything else means **STOP — do not trade**.

Stage 3 — Paper-trading soak (weeks) [YOU RUN]

Run the worker in paper mode and watch for **weeks**:

- Orders fill and the equity curve is sane.
- The **circuit breaker actually trips** on a drawdown day (the risk engine's `sync_account()` must be fed equity on a 30–60s timer).
- `reconcile_positions()` shows **no drift** vs the broker.
- No crashes or stuck consumers; alerts fire as expected.

Stage 4 — Tiny live, supervised [YOU RUN]

- Start at **\$100** with hard caps and manual supervision.
- **Rehearse the kill switch before funding:** `emergency_flatten()` cancels open orders, flattens positions, and halts (unit-tested — rehearse it live).
- Set `TRADING_MODE=live` only at this stage; scale only if results match expectations.

Emergency procedures

- **Halt + flatten:** call `emergency_flatten()` (or `halt()` to just stop new orders).
- **Drawdown auto-stop:** the circuit breaker trips at `circuit_breaker_loss_pct` (default 7% daily) once `sync_account()` is feeding it — verify in Stage 3.

Operational glue: the worker already runs `risk.run_account_sync(...)` on a timer (feeding the drawdown monitor / circuit breaker) and the manual-order path is wired end-to-end. Small remainder: a `risk.reset_daily()` call at session open, to verify in the Stage 3 soak.